1-Introduction

1.1 Overview:

The design of integrated circuits (ICs) follows a structured VLSI (Very Large Scale Integration) design flow. Modern IC design heavily relies on **hierarchical design methodology**, where complex circuits are built from smaller, reusable building blocks called **cells**. This approach significantly improves design productivity, reduces errors, and enables design reuse across multiple projects. Each cell contains its own schematic, symbol, and layout, and can be instantiated multiple times within a larger circuit.

Cadence Virtuoso is an industry-leading Electronic Design Automation (EDA) platform that supports hierarchical design through its **Library-Cell-View** structure. Designers can create basic gates (inverters, NAND, NOR) as individual cells, then use their **symbols** to build more complex circuits at the top level.

1.2 CMOS Technology and 45nm Node:

CMOS (Complementary Metal-Oxide-Semiconductor) technology is the foundation of modern digital circuit design, offering low power consumption, high noise immunity, and excellent scalability. This project utilizes the **gpdk045** 45nm technology library, which provides accurate device models, design rules, and parasitic data for the 45nm process node. The 45nm technology enables high-density integration, reduced power consumption, and high-speed operation.

1.3 Hierarchical Design Using Symbols:

In hierarchical design, each logic gate (e.g., inverter, NAND) is first designed at the transistor level as a **schematic** and **symbol**. The **symbol** serves as an abstraction of the gate, hiding its internal transistor details and allowing designers to focus on higher-level connectivity

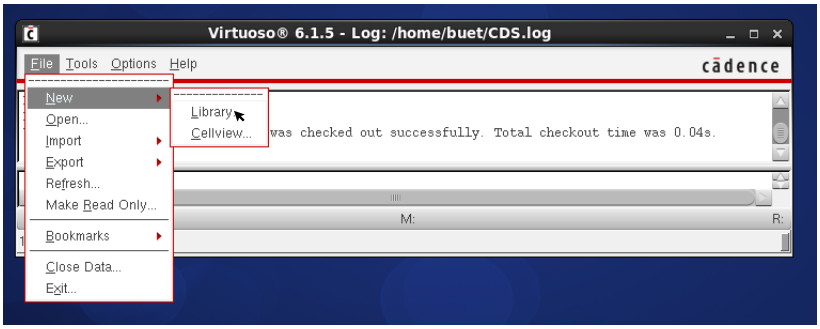
1.4 Project Objectives:

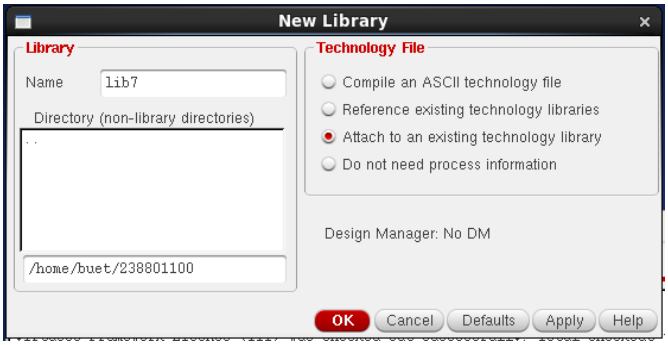
1-Library Creation: Create a new library (lib7) and attach the gpdk045 technology file.
2-Inverter Cell Design: Design the schematic and symbol of a CMOS inverter in lib7.
3-NAND Gate Integration: Use previously designed NAND gate symbols from lib2
4-Top-Level Schematic: Build the complete logic circuit by instantiating inverter and NAND gate **5-Design:** Create a physical layout of the complete circuit.
6-Physical Verification: Passed DRC and LVS verification, confirming manufacturability and electrical correctness.

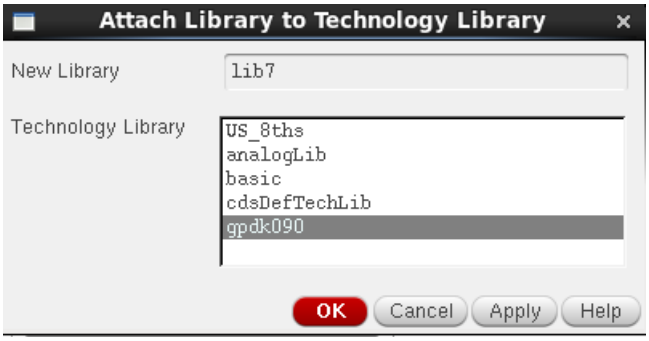
2-Schematic Design

2.1 Creating the Library:

The first step was to create a new library named **"lib7"** in the CIW (Command Interpreter Window) by clicking **File → New → Library**. The technology file **"gpdk090"** was attached to ensure all design rules and device models are available.

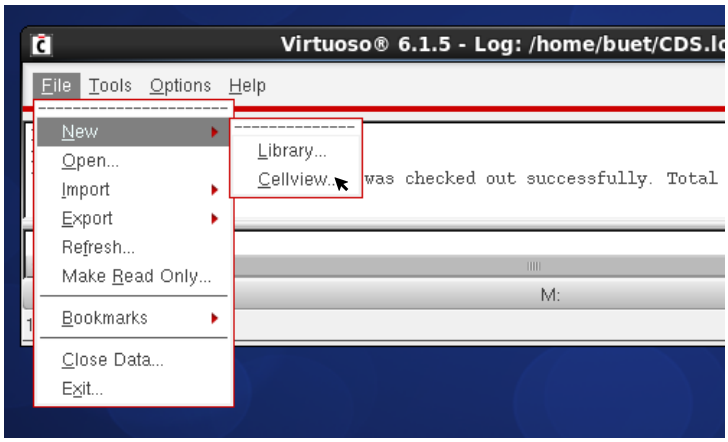


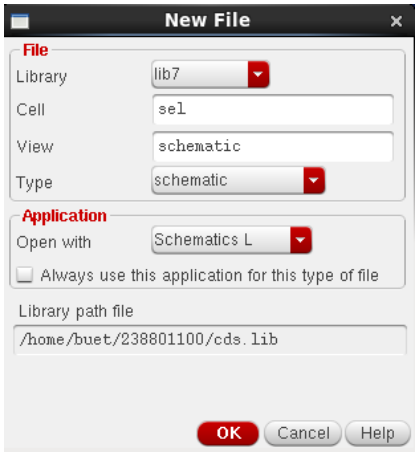




2.2 Creating the Schematic:

clicked **File → New → CellView**, named the cell **"sel"**, and selected **"schematic"** as the view type. The schematic editor opened, allowing me to place and connect components.



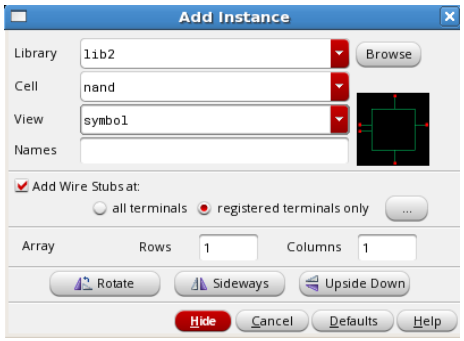


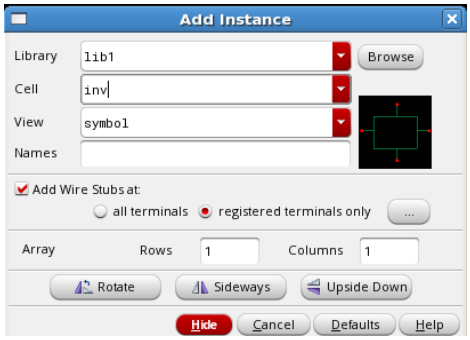
Creating Symbols:

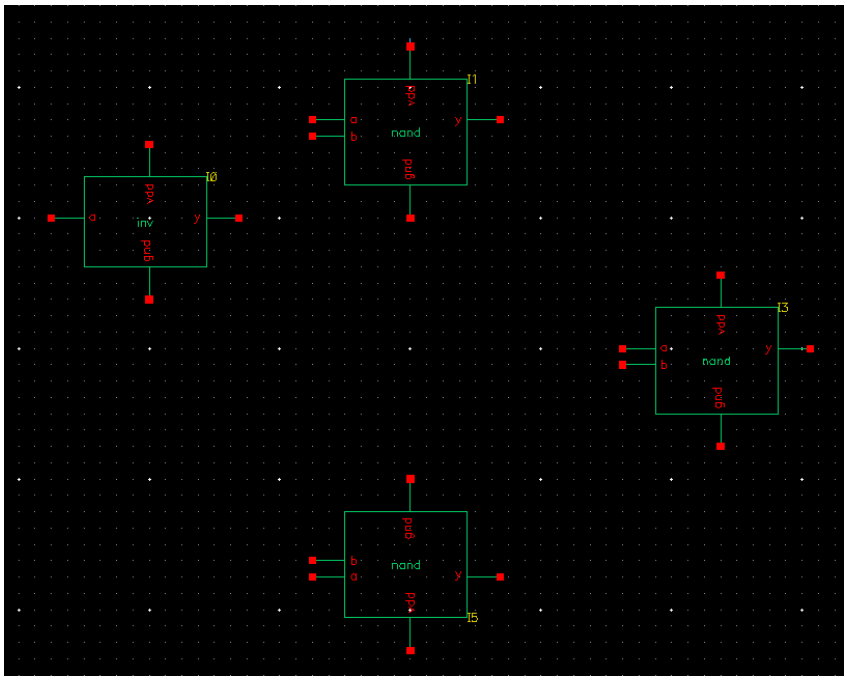
To build the top-level circuit, I pressed the **'I'** key to open the Add Instance window and instantiated the following symbols:

1x Inverter from lib1

3x NAND Gate from lib2

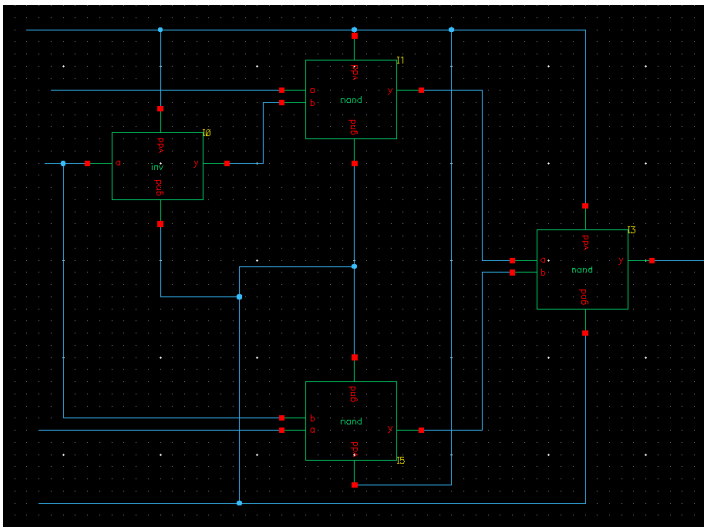






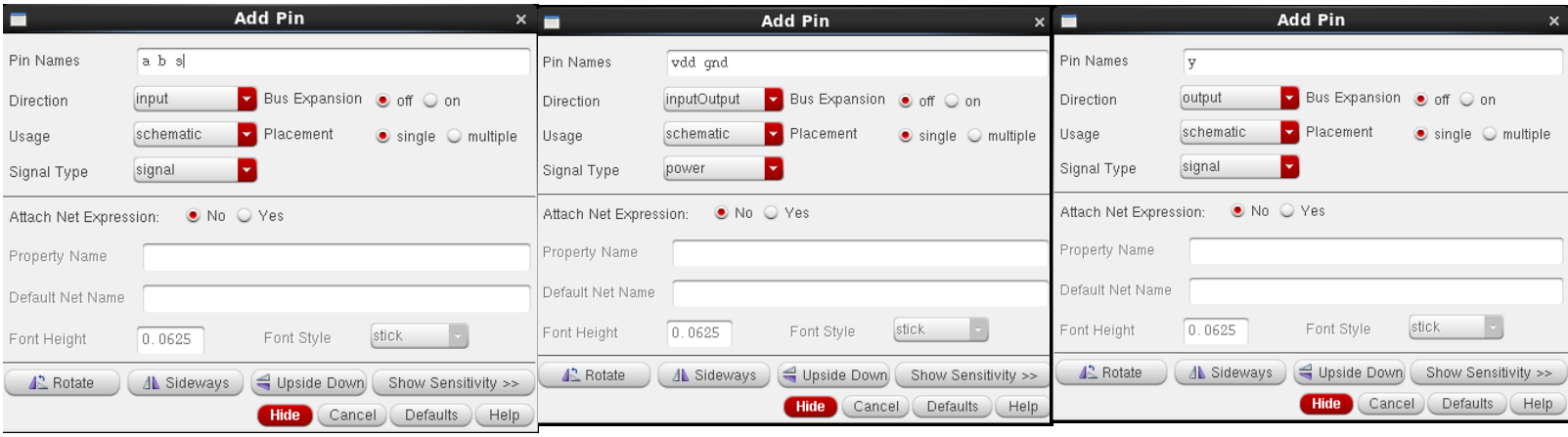
2.4 Wiring the Circuit:

After placing all symbols in the schematic, I pressed the **'W'** key to open the Add Wire window and connected the components according to the circuit design. The connections were made as follows:



2.5 Adding Pins:

To enable the top-level circuit to interface with external signals, I pressed the **'P'** key to add pins:

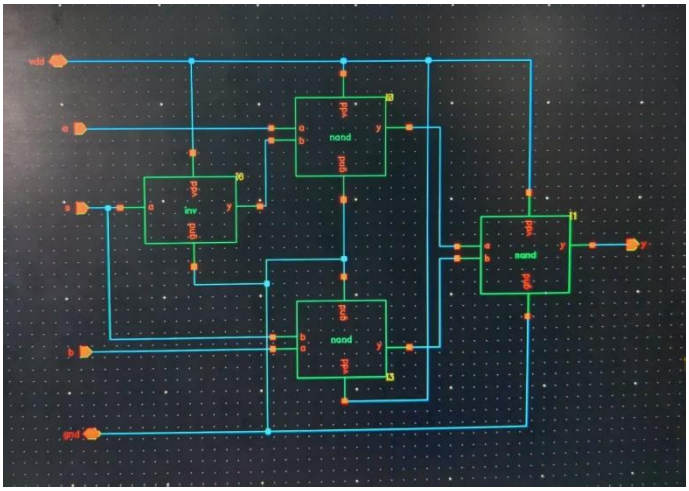


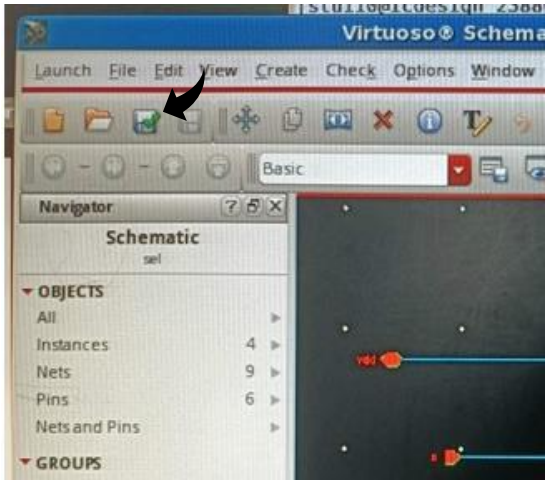
"a b c" pin with direction set to **Input**
"vdd gnd" pin with direction set to **Input output**
"y" pin with direction set to **Output**

After entering the pin names, I clicked **"Hide",** placed the pins in the schematic, and wired them to their respective connections.

2.6 Check and Save:

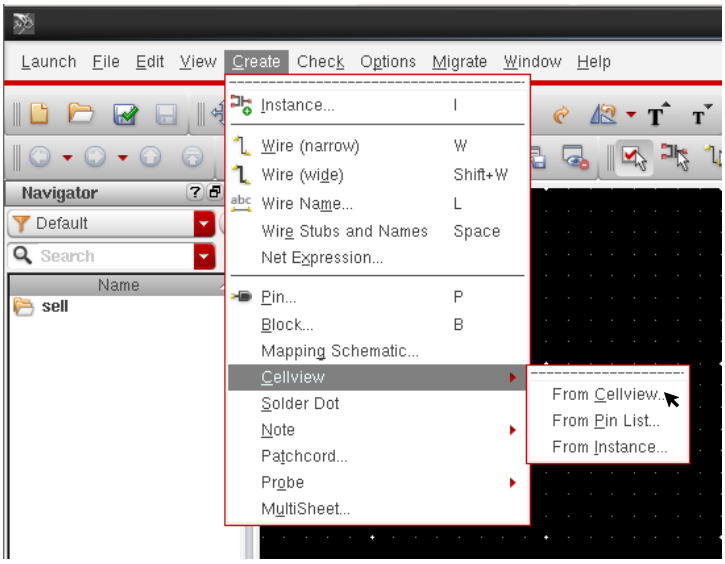
After completing the schematic, I clicked the "Check and Save" button to verify the design and save the cell view

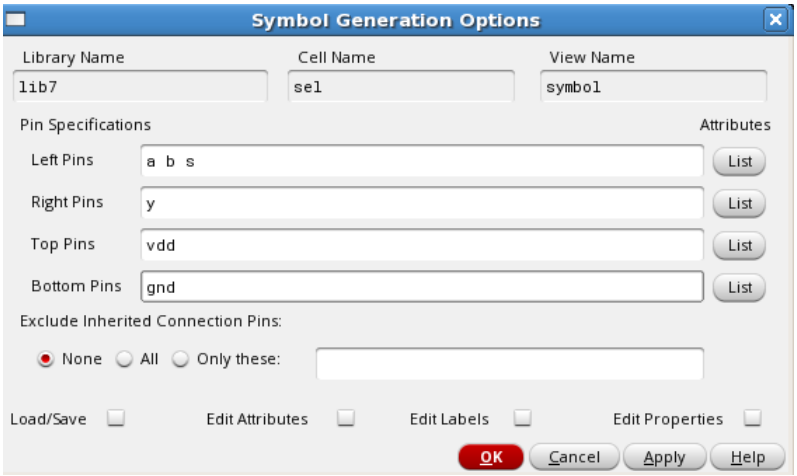


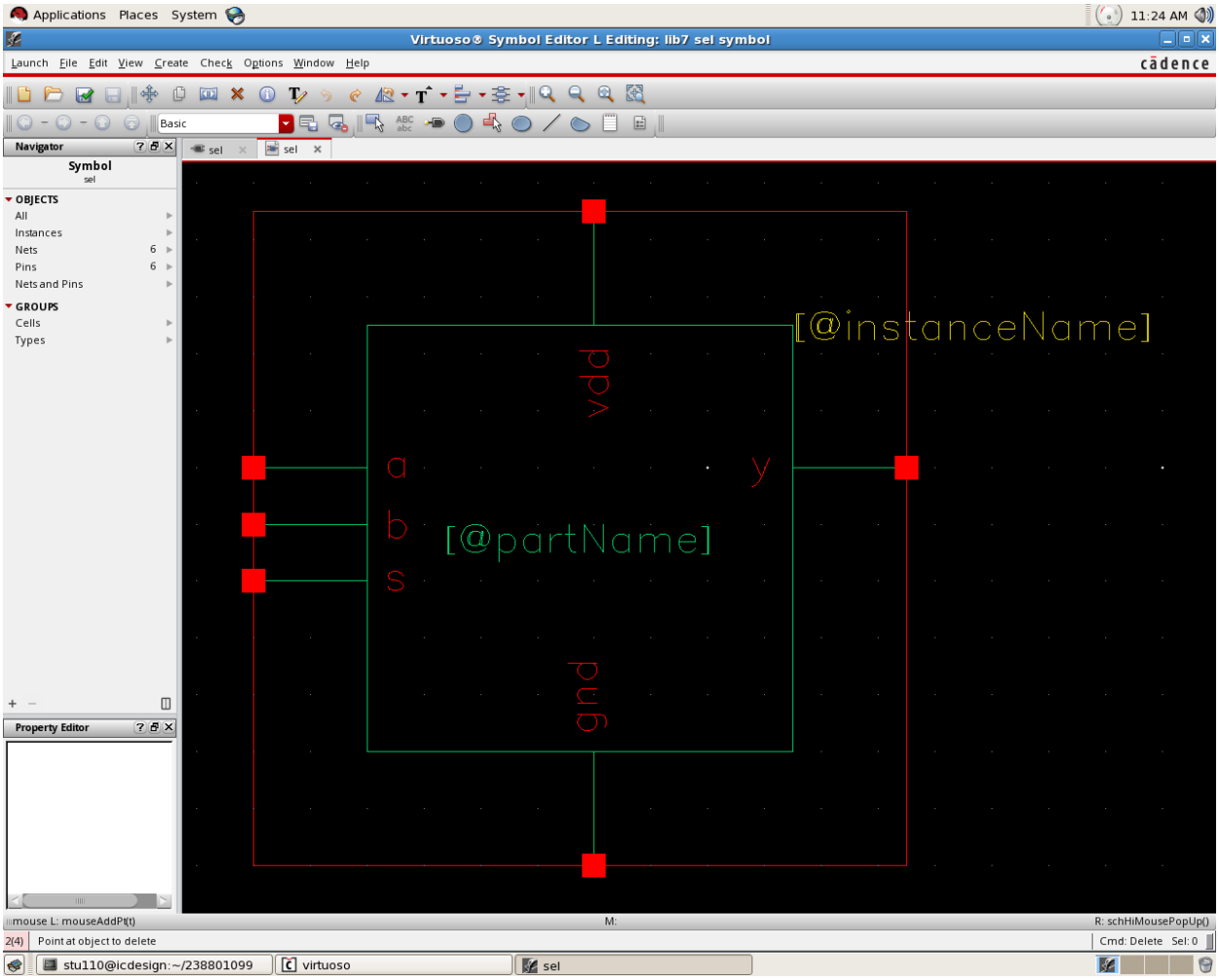


2.7 Creating the Symbol:

A symbol was generated for the top-level circuit using **Create → Cellview → From Cellview.**



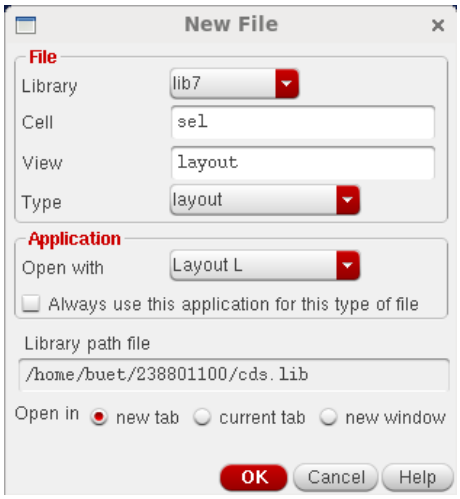


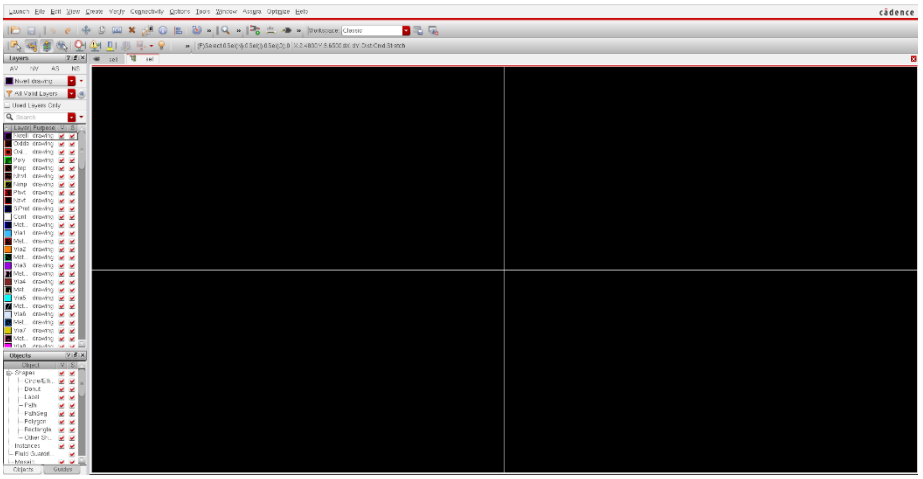


5. Layout Design

5.1 Creating the Layout View:

To create the layout, clicked **File** → **New** → I selected the **lib7** library and my cell from the list, then., I set the View to "**layout**" and selected "**layout**" as the Type, then clicked **OK** to open the layout editor





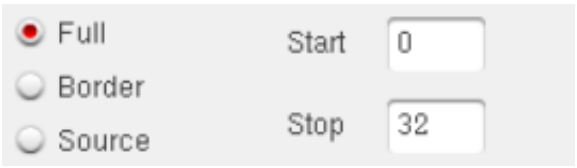
5.2Setting the Display Options:

Before starting the layout design, I configured the display options I pressed the **'E'** key on the keyboard to open the **Display Options** window. set the **Start** level to **0** and **Stop** level to **32** in the array display section



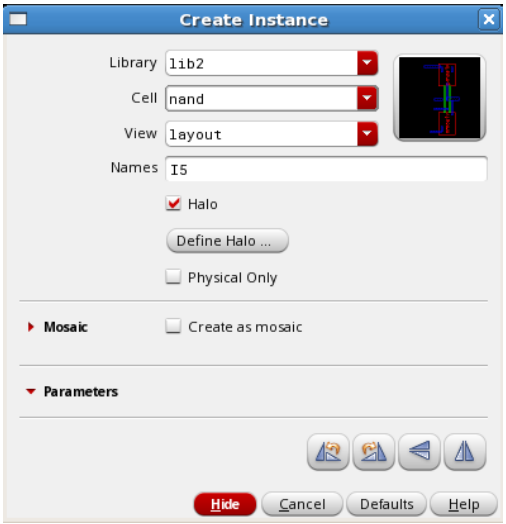


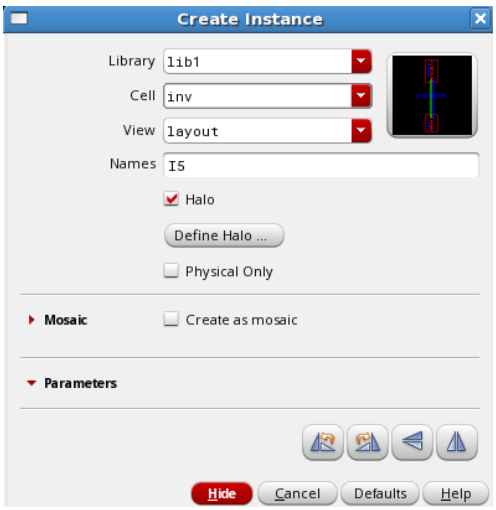




5.3Create instance in the Layout:

After configuring the display options, I pressed the **'I'** key on the **keyboard** to open the Create Instance window. In the dialog box, I set the **Library** to **lib2**, the Cell to **nand**, and pressed the **'I'** key again to instantiate the **inv** cell. In the Create Instance window, I set the **Library** to **lib1**, the **Cell** to **inv**, and the **View** to **layout**.

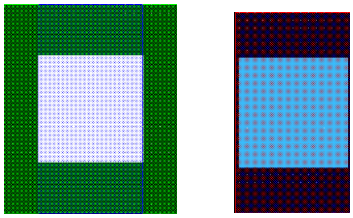


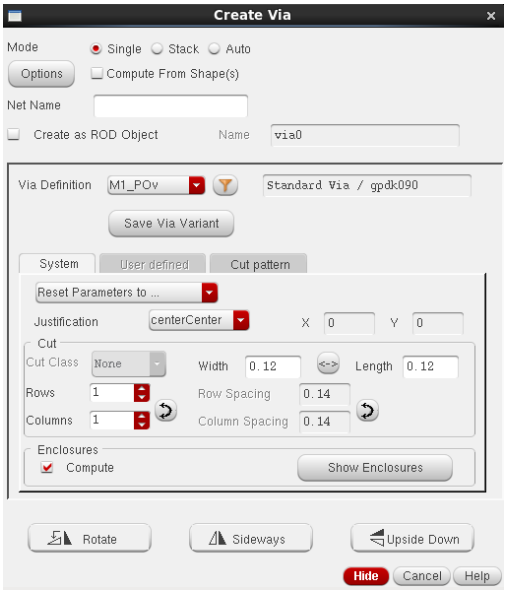


5.4 Routing the Cell Connections:

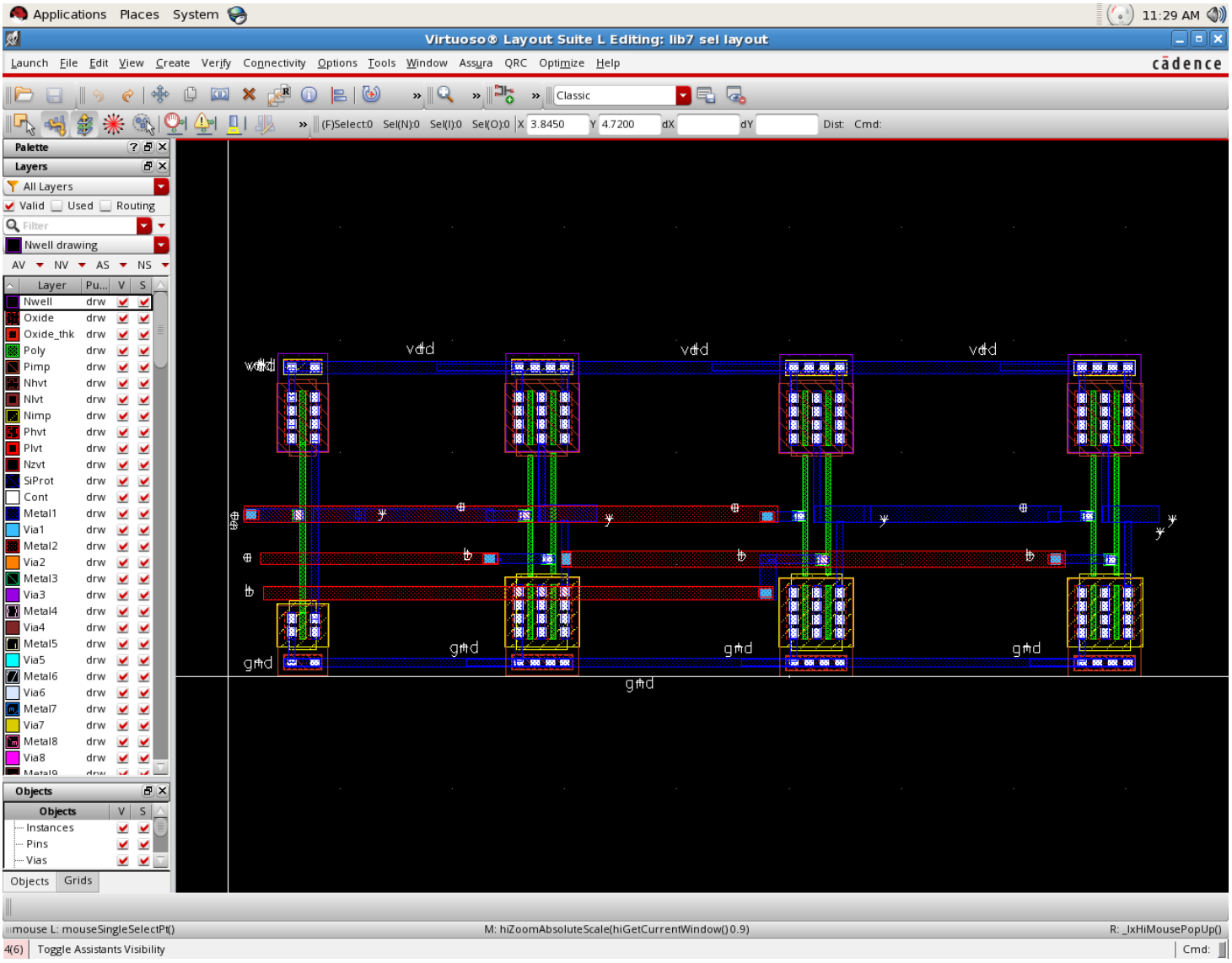
For the routing, I used **Metal1** as the primary layer for most signal and power connections. Where additional routing resources were needed, I used **Metal2** to avoid congestion and ensure proper connectivity. For gate connections and short-distance routing within the cells, I used the **Poly** layer After completing the routing,. I pressed the **'O'** key on the keyboard to open the **Create Via** window. In the dialog box, I selected the **M1_PO** for Poly-to-Metal1 connections or **M1_M2** for Metal1-to-Metal2 connections







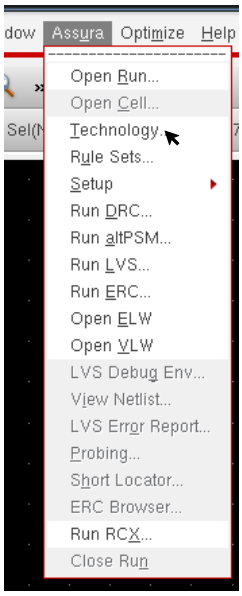
5.5Layout View:



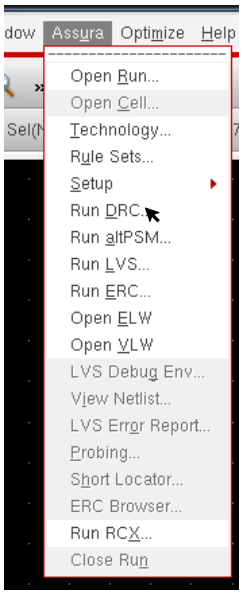
5.6Running DRC:

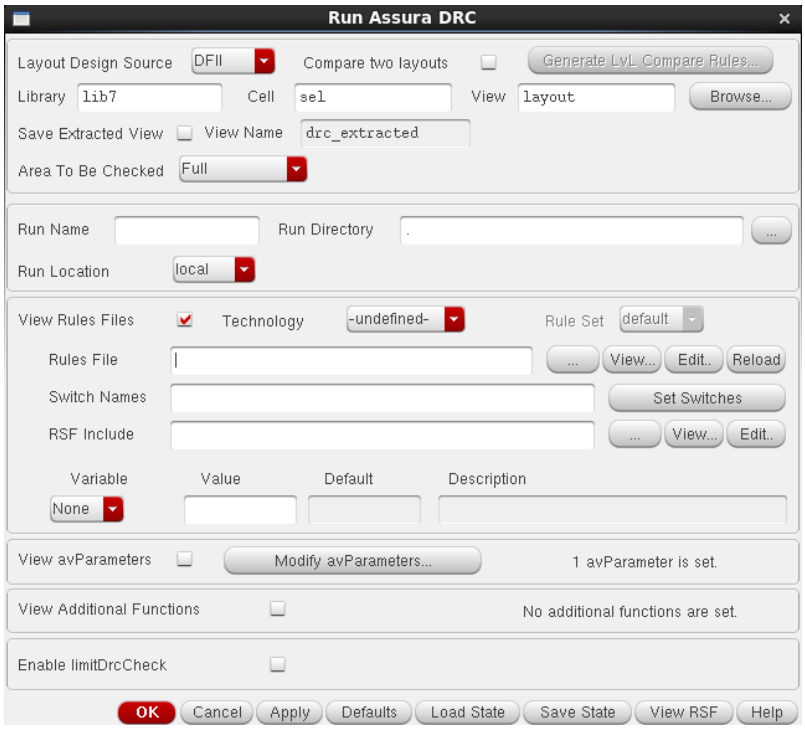
To run DRC, I went to the Virtuoso layout menu bar and clicked Assura → Technology. In the "Assura Technology File" dialog box, I entered the path to the technology file: **/home/csu1/pdk/gpdk045_v5/assura_tech.lib** and clicked OK.

Next, I clicked **Assura → Run DRC** from the menu bar. In the "Run Assura DRC" window, I switched the **Technology** option to **"gpdk045_av"** and verified that all other settings were correct. I then clicked **OK** to run the DRC checker.









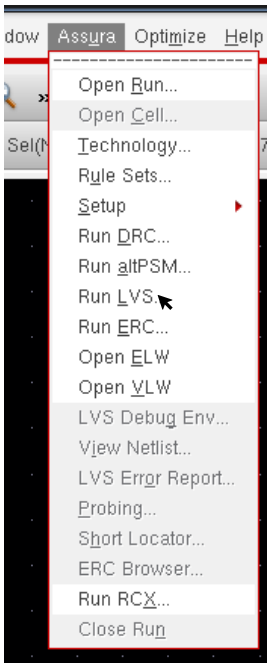
5.7 DRC Results:

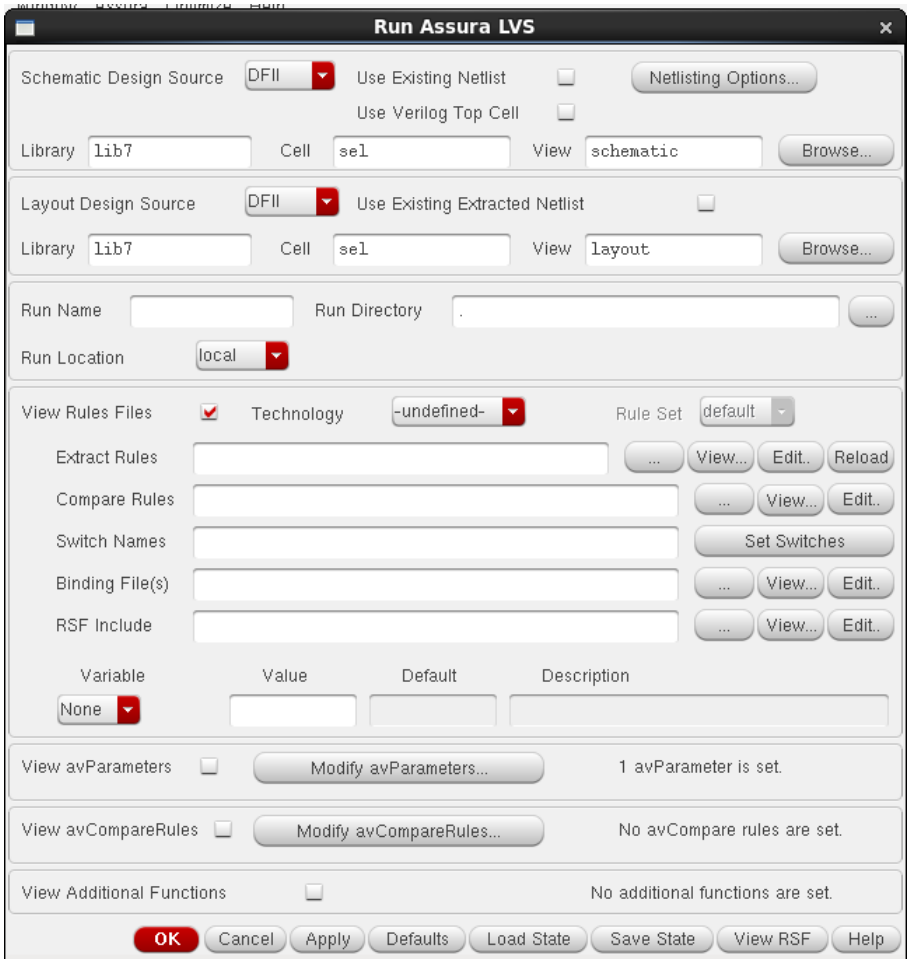
After running the **DRC** checker, the results showed that **no errors** were found in the layout. The design **successfully passed** all design rule checks, confirming that the layout complies with the 45nm manufacturing constraints.



5.8 Running LVS:

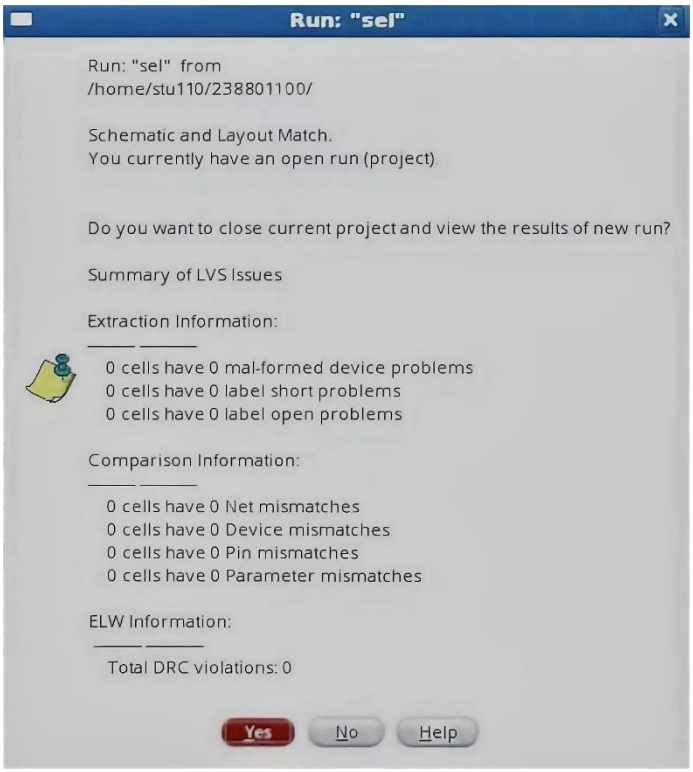
To run **LVS**, I went to the Virtuoso layout menu bar and clicked **Assura → Run LVS**. In the "**Run Assura LVS**" window, I switched the Technology option to "gpdk045" and verified that all other settings were correct. I then clicked **OK** to run the **LVS** checker.





5.9 LVS Results:

After running the LVS checker, the comparison confirmed that the layout matches the schematic exactly. The results showed zero net mismatches, zero device mismatches, zero pin mismatches, and zero parameter mismatches. Additionally, no mal-formed device problems, label short problems, or label open problems were found. The successful LVS verification confirms that the physical layout correctly implements the circuit design and is electrically identical to the schematic



6.Conclusion:

In this project, I successfully designed a **CMOS** digital circuit using hierarchical design methodology with **Cadence Virtuoso** in **45nm technology**. The design process began with creating an **inverter** in **lib1** and integrating **NAND gate** symbols from **lib2** into a top-level logic circuit. The schematic and symbol were completed for all cells and the top-level circuit, and pre-layout simulation verified the circuit functionality through transient analysis**. A DRC**-clean and **LVS**-correct layout was then developed, confirming manufacturability and electrical correctness. Parasitic extraction was performed to extract parasitic components from the layout, and post-layout simulation was conducted to compare performance degradation due to parasitics. This project demonstrates the power of hierarchical design methodology, where complex circuits can be efficiently built from smaller, reusable cells, which is essential for modern IC design as it enables design reuse, faster development cycles, and reduced errors.